Chatbot Abuse & Cost Protection

A tiered plan for deploying the public AI chat widget safely

Project: `mcp_chatbot` (Odoo addon + FastAPI sidecar) • Prepared 11 June 2026

The core problem in one sentence. The chat widget is public: any visitor — even one who never logs in — can send messages, and every message calls a paid AI service that **you** are billed for. Today there is nothing in the code that limits how many messages a single person (or a script) can send. As shipped, the system trusts users not to abuse it. This report explains, tier by tier, the specific risk and the fix.

How a message becomes a cost

When a visitor opens the website, the browser is handed a one-hour access pass (a signed token). With that pass it sends messages straight to the chat server, which calls the AI model (Groq) and any connected tools. **You are billed by the volume of text** processed. The only checks today are: the message isn't empty, and the bot is switched "online". No speed limit, no size limit, no spending limit exists anywhere. Each tier below closes one gap in that chain.

TIER 0 · REQUIRED Basic limits inside your own code

PROBLEM IT SOLVES

Nothing stops one visitor from sending thousands of messages in a loop, pasting an entire book into a single message, or quietly running up a huge bill over a day. A determined person could even use your bot as a **free AI service** for their own projects — on your bill.

SOLUTION

Three small rules added directly to the chat server:

- **Speed limit (rate limiting).** e.g. "1 message every few seconds, 30 per conversation." A real person never notices; a flooding script is blocked after the first attempt. A ready-made tool (`slowapi`) does this in a few lines.
- **Maximum message length.** Reject anything over, say, 2,000 characters — so no one can make you pay to process a pasted book. One line of code.
- **Daily spending kill-switch.** Track usage as it happens; when the day's budget is reached, the bot switches itself off until tomorrow. This reuses the on/off "status" switch the project already has.

Coffee-machine view: the machine pours one cup at a time, refuses buckets, and shuts off once the day's coins hit a set amount.

Effort: a few hours. **Verdict:** do not deploy without this — it stops the large majority of abuse on its own.

TIER 1 · BEST VALUE A bouncer at the edge (Cloudflare)

PROBLEM IT SOLVES

Even with code-level limits, attack traffic still reaches your server before it gets rejected — using up your server's capacity, and in some cases costing money before the limit kicks in. Large automated floods can also simply overwhelm the machine.

SOLUTION

Place a free service (**Cloudflare**) **in front of** your server. All internet traffic hits Cloudflare first; it recognizes bots, attack scripts, and floods and blocks them **before they ever reach your server** — meaning before they cost you anything. Genuine visitors pass straight through and notice nothing. This is mostly configuration, not programming.

Coffee-machine view: a bouncer on the sidewalk stops obvious troublemakers and robots before they touch the button; real customers walk right past.

Effort: very low (mostly setup). **Verdict:** biggest protection for the least work — if you do only one thing, do this.

TIER 2 · IF NEEDED A quiet "prove you're human" check

PROBLEM IT SOLVES

Sophisticated bots can imitate real browsers and slip past the limits above, especially when targeting the anonymous (not-logged-in) path where there is no account to hold them accountable.

SOLUTION

Before a **stranger** sends their first message, require a quick "are you a robot?" check. The modern version (**Cloudflare Turnstile**) is usually **invisible** — a real person sees nothing and just chats; a bot is silently stopped. It is not the old "click all the traffic lights" puzzle.

Coffee-machine view: a small turnstile a person steps through without thinking, but a robot arm can't get past.

Effort: low–medium. **Verdict:** add only if abuse continues after Tiers 0 and 1 — don't add friction for real visitors before you need to.

TIER 3 · SKIP FOR NOW

Content filtering & advanced quotas

PROBLEM IT SOLVES

People may try to trick the bot into saying offensive things under your brand (“jailbreaking”), or you may eventually want fine-grained per-account usage budgets.

SOLUTION

AI content-moderation filters on the bot’s output, plus per-user quotas. Real, but **premature for a first launch** — solve the money and flooding problems first and revisit this only if it becomes a real issue.

Effort: high. **Verdict:** not part of v1.

Summary at a glance

Tier	What it is	Main problem it solves	Effort	Verdict
0	Speed limit, message-size cap, daily spend kill-switch (in your code)	One person flooding messages or running up the bill	Few hours	Required before launch
1	Cloudflare bouncer in front of the server	Automated floods reaching your server at all	Mostly config	Best value — do this
2	Invisible “are you human?” check for strangers	Advanced bots imitating real browsers	Low–medium	Only if abuse continues
3	Content filters & per-account quotas	Brand-damaging jailbreaks; fine-grained budgets	High	Skip for v1

Bottom line

Real products do **not** rely on users’ goodwill — that is simply where this code stands today. The practical path is to put up two cheap fences before going live: **Tier 0** (limits in the code) and **Tier 1** (the free Cloudflare bouncer). Together they handle the overwhelming majority of abuse. Tiers 2 and 3 are held in reserve and added only if real-world abuse demands them.

Related hardening note (outside the tiers): keep the AI provider key secret and rotate it if it is ever exposed — a leaked key is the same “unlimited spend” risk through a different door.